

## Contents

[sercon manual](#)

[Table of contents](#)

[1. Overview](#)

[2. Quickstart](#)

[3. Library API — pkg/scriptengine](#)

[Engine, Options, New](#)

[Registering bindings](#)

[Run, RunFile](#)

[WriteTypes](#)

[PromisifyAsync\[T\] and Promised\[T\]](#)

[Version](#)

[4. CLI — sercon](#)

[5. Built-in script api](#)

[6. JavaScript runtime built-ins \(goja\)](#)

[Globals](#)

[Collections](#)

[Typed arrays](#)

[Iteration and generators](#)

[Not \(yet\) provided](#)

[7. Async runtime additions \(goja\\_nodejs\)](#)

[Timers](#)

[console](#)

[require](#)

[8. TypeScript support](#)

[9. Top-level await](#)

[10. Module resolution](#)

[11. Timeouts and cancellation](#)

[12. Error semantics](#)

[13. Type generation \(.d.ts\)](#)

[14. Limitations and gotchas](#)

# sercon manual

---

Long-form reference for the `pkg/scriptengine` library, the `sercon` CLI, the built-in script API, and the JavaScript runtime surface that `scriptengine` exposes via [goja](#) and [goja\\_nodejs](#). Examples are runnable — save snippets as `.ts` files and execute with `sercon file.ts` unless otherwise noted.

---

# Table of contents

---

1. [Overview](#)
  2. [Quickstart](#)
  3. [Library API](#) — `pkg/scriptengine`
  4. [CLI](#) — `sercon`
  5. [Built-in script](#) `api`
  6. [JavaScript runtime built-ins](#) (`goja`)
  7. [Async runtime additions](#) (`goja_nodejs`)
  8. [TypeScript support](#)
  9. [Top-level](#) `await`
  10. [Module resolution](#)
  11. [Timeouts and cancellation](#)
  12. [Error semantics](#)
  13. [Type generation](#) (`.d.ts`)
  14. [Limitations and gotchas](#)
- 

## 1. Overview

---

`sercon` is a Go program that runs TypeScript files. It's built from two parts:

- `pkg/scriptengine` — a library you embed in your own Go code. You register Go-callable bindings, then call `Run` / `RunFile` to execute a `.ts` script that talks to them.
- `cmd/sercon` — a thin CLI built on the library, useful for ad-hoc test scripts and as a working example of every binding kind.

The runtime is pure Go: `goja` is the JS engine, `esbuild` (used as a Go library) is the TS→JS transpiler, and `goja_nodejs` provides Promises, `setTimeout`, `console`, and CommonJS `require`. No cgo, no Node.

## 2. Quickstart

---

Install:

```
go install github.com/codedeviate/sercon/cmd/sercon@latest
```

Run one of the bundled examples:

```
sercon examples/scripts/smoke.ts examples/scripts/async.ts
```

Generate a declaration file for editor autocomplete:

```
sercon --emit-dts api.d.ts
```

Embed in your own Go program:

```
eng := scriptengine.New(scriptengine.Options{
    Timeout:    5 * time.Second,
    ScriptRoot: "./scripts",
})
eng.Register("upper", strings.ToUpper)
val, err := eng.RunFile(ctx, "scripts/main.ts")
```

### 3. Library API — `pkg/scriptengine`

#### Engine, Options, New

```
type Options struct {
    Timeout      time.Duration // wall-clock limit per Run; 0 disables
    ScriptRoot   string        // base for require/import resolution
    DisableConsole bool          // turn off the goja_nodejs console module
}

func New(opts Options) *Engine
```

`Engine` is the host of all registered bindings. Construct it once, then call `Run` / `RunFile` as many times as needed — each call gets a *fresh* `*goja.Runtime` so script state never leaks between invocations.

`ScriptRoot` defaults to the working directory at `New` time if left empty. Both `Timeout` and `ScriptRoot` can be overridden on a per-Run basis only by constructing a fresh `Engine`; see [Out-of-scope](#) for the per-Run override on the roadmap.

#### Registering bindings

| Function                                                     | Use when...                                                                                                                                             |
|--------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>Register(name, value)</code>                           | The binding is a pure function, struct, or primitive that doesn't need access to the runtime.                                                           |
| <code>RegisterNamespace(name, members map[string]any)</code> | You want to expose <code>name.x</code> , <code>name.y</code> without defining a Go struct.                                                              |
| <code>RegisterConstructor(name, ctor)</code>                 | The binding is a Go function whose return value should be treated as a class in the emitted <code>.d.ts</code> . (Runtime semantics today are identical |

| Function                                          | Use when...                                                                                                       |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
|                                                   | to Register ; the d.ts emitter is the only differentiator.)                                                       |
| RegisterFactory(name, factory func(vm, loop) any) | The binding needs the per-Run *goja.Runtime or *eventloop.EventLoop in scope (typical for Promise-returning I/O). |
| RegisterNamespaceFactory(name, factory)           | Same, but the factory returns the full member map.                                                                |

Example — synchronous function binding:

```
eng.Register("greet", func(name string) string { return "hi " + name })
```

Example — namespace with a sync member:

```
eng.RegisterNamespace("math2", map[string]any{
    "square": func(n float64) float64 { return n * n },
    "pi":     3.14159,
})
```

Example — async (Promise-returning) binding via PromisifyAsync :

```
eng.RegisterFactory("httpGet", func(vm *goja.Runtime, loop
*eventloop.EventLoop) any {
    return scriptengine.PromisifyAsync(vm, loop,
        func(ctx context.Context, call goja.FunctionCall) (map[string]any,
error) {
            url := call.Argument(0).String()
            resp, err := http.Get(url)
            if err != nil {
                return nil, err
            }
            defer resp.Body.Close()
            body, _ := io.ReadAll(resp.Body)
            return map[string]any{"status": resp.StatusCode, "body":
string(body)}, nil
        })
})
```

**Run, RunFile**

```
func (e *Engine) Run(ctx context.Context, name, source string) (goja.Value,
error)
func (e *Engine) RunFile(ctx context.Context, path string) (goja.Value,
error)
```

`Run` executes `source` as an entry-script TS. `name` is used in stack traces. The returned value is currently always `undefined` — top-level expression capture is on the backlog.

Both methods respect `Options.Timeout` and `ctx` cancellation, whichever fires first; the resulting error is either `ErrScriptTimeout`, `ctx.Err()`, or the underlying JS exception.

## WriteTypes

```
func (e *Engine) WriteTypes(w io.Writer) error
```

Emits a `.d.ts` describing the registered surface. See section [13. Type generation](#) for what the mapping looks like.

## PromisifyAsync[T] and Promised[T]

```
func PromisifyAsync[T any](vm *goja.Runtime, loop *eventloop.EventLoop,
    work func(ctx context.Context, call goja.FunctionCall) (T, error),
) func(goja.FunctionCall) goja.Value

type Promised[T any] func(call goja.FunctionCall) goja.Value
```

`PromisifyAsync` turns blocking Go work into a JS Promise. It launches the work in a goroutine, parks a `SetTimeout` sentinel so `loop.Run` doesn't return early, and schedules the resolve/reject back onto the event loop. **Required** for any Promise-returning binding — `RunOnLoop` alone is not counted as a live job by the event loop.

`Promised[T]` is a marker type the `.d.ts` emitter recognises so it can emit `Promise<T>` rather than `unknown`. Today it's a typed wrapper that behaves like a plain function at runtime; using it is optional.

## Version

```
import "github.com/codedeviate/sercon/pkg/scriptengine"

fmt.Println(scriptengine.Version) // "0.1.0"
```

Bumped in lockstep with the git tag.

## 4. CLI — `sercon`

---

```
sercon [flags] <script.ts> [script.ts ...]
sercon --examples | --help | --version
```

| Flag                                  | Purpose                                                                                       |
|---------------------------------------|-----------------------------------------------------------------------------------------------|
| <code>-timeout DURATION</code>        | Wall-clock limit per script (default <code>10s</code> ; <code>0</code> disables).             |
| <code>-root DIR</code>                | Override the script root for <code>require</code> / <code>import</code> resolution.           |
| <code>-emit-dts PATH</code>           | Write the example bindings' <code>.d.ts</code> to <code>PATH</code> and exit.                 |
| <code>-v</code>                       | Verbose: log timing on failures.                                                              |
| <code>-h</code> , <code>--help</code> | In-depth colorized help.                                                                      |
| <code>--examples</code>               | In-depth colorized walkthrough of every feature.                                              |
| <code>--version</code>                | Print the engine version (plus <code>goja</code> / <code>esbuild</code> build-info versions). |

Exit codes:

- `0` — every script passed.
- `1` — at least one script threw, timed out, failed to parse, or the CLI's own argument parsing failed.

The CLI registers a single `api` namespace; see the next section.

## 5. Built-in script `api`

---

The CLI exposes the following globals to every script:

```

declare const api: {
  log(...args: unknown[]): void;

  assert: {
    equal(actual: unknown, expected: unknown, msg?: string): void;
    ok(cond: unknown, msg?: string): void;
  };

  http: {
    get(url: string): Promise<{ status: number; body: string }>;
    post(url: string, body?: string): Promise<{ status: number; body:
string }>;
  };

  time: {
    nowMs(): number;
    sleep(ms: number): Promise<void>;
  };

  env: {
    get(name: string): string | undefined;
  };
};

```

Examples:

```

api.log("hello", 1 + 2);
api.assert.equal(api.time.nowMs() > 0, true);

const r = await api.http.get("https://example.com");
api.assert.ok(r.status === 200, `expected 200, got ${r.status}`);

await api.time.sleep(50);
const home = api.env.get("HOME") ?? "(none)";

```

HTTP bindings use `net/http` with a 5-second default per-request timeout and surface real `Promise<...>` values through the event loop. They are *not* mockable from JS — they go to the real network.

## 6. JavaScript runtime built-ins (goja)

`scriptengine` runs on goja, which implements **ES5.1 + a large subset of ES6+**. The following are present and behave per spec:

### Globals

- `Object`, `Array`, `String`, `Number`, `Boolean`, `BigInt`
- `Math`, `Date`, `JSON`, `RegExp`
- `Error`, `TypeError`, `RangeError`, `SyntaxError`, `ReferenceError`, `EvalError`, `URIError`
- `Promise` (provided via goja's native implementation; resolution microtasks are pumped by the event loop)
- `Symbol` (partial: well-known symbols `Symbol.iterator`, `Symbol.asyncIterator`, etc., are supported)
- `Proxy`, `Reflect` (partial)

```
api.log(Math.PI.toFixed(4), Math.max(1, 9, 3));
api.log(new Date().toISOString());
api.log(JSON.stringify({ a: 1, b: [2, 3] }));
api.log("abc".repeat(3), "abc".padStart(6, "_"));
```

## Collections

- `Map`, `Set`, `WeakMap`, `WeakSet`

```
const counts = new Map<string, number>();
for (const ch of "banana") counts.set(ch, (counts.get(ch) ?? 0) + 1);
api.log([...counts]); // [{"b",1}, {"a",3}, {"n",2}]
```

## Typed arrays

- `ArrayBuffer`, `DataView`
- `Int8Array`, `Uint8Array`, `Uint8ClampedArray`, `Int16Array`, `Uint16Array`, `Int32Array`, `Uint32Array`, `Float32Array`, `Float64Array`

```
const buf = new ArrayBuffer(4);
new DataView(buf).setUint32(0, 0xCAFEBAFE);
api.log(new Uint8Array(buf)[0].toString(16)); // ca
```

## Iteration and generators

- `for...of`, `for...in`, `spread`, `destructuring`
- Generator functions (`function*`)
- Async generators (`async function*`) — supported via goja's event loop



```
function* fib() {
  let [a, b] = [0, 1];
  while (true) {
    yield a;
    [a, b] = [b, a + b];
  }
}
const it = fib();
const first10 = Array.from({ length: 10 }, () => it.next().value);
api.log(first10);
```

## Not (yet) provided

- `fetch` — use `api.http.*` from the CLI, or register your own binding.
- `Worker`, threading primitives.
- DOM globals (`window`, `document`).
- Native ES modules (`import` / `export` are *transpiled* to CommonJS at load time — see [section 8](#)).

## 7. Async runtime additions (goja\_nodejs)

The event loop bundles a small Node-compatible runtime on top of goja:

### Timers

```
setTimeout(() => api.log("after 50ms"), 50);
setInterval(() => api.log("tick"), 100);
setImmediate(() => api.log("next tick"));
clearTimeout(t); clearInterval(i); clearImmediate(im);
```

The event loop holds the script alive while *any* timer is pending. If your script ends with only `setTimeout` callbacks scheduled, the run will wait for them to fire before returning.

### console

```
console.log("info");
console.error("oops");
console.warn("careful");
console.info("fyi");
console.debug("trace");
```

Disable with `Options.DisableConsole = true` if you want a clean global namespace.

## require

CommonJS `require`, with TS-aware extension fallback added by `sercon` (see [section 10](#)):

```
const helpers = require("./helpers");
helpers.doThing();
```

Both `require("./foo")` and `import { x } from "./foo"` resolve to the same module instance per Run; esbuild rewrites `import` to `require` at transpile time.

## 8. TypeScript support

TS is transpiled by esbuild in-process. There is no `tsc`, no `tsconfig.json`, no type-checking — types are erased and the resulting JS is what executes. Practical implications:

- All TS syntax esbuild supports is allowed: type annotations, `interface`, `type`, generics, enums (compiled to objects), namespaces, decorators (legacy & TC39 stage 3).
- `import type { X } from "y"` is stripped entirely (no runtime require).
- TS errors are *not* surfaced as build errors — only esbuild parse errors are. Run a separate `tsc --noEmit` in CI if you want type enforcement.
- `tsconfig.json` is **not** consulted.

`.tsx` and `.jsx` loaders are wired but not yet exercised end-to-end; see `OUT-OF-SCOPE.md`.

## 9. Top-level `await`

Top-level `await` works in entry scripts:

```
const r = await api.http.get("https://example.com");
api.log(r.status);
```

How it works under the hood: esbuild rejects top-level `await` with the `cjs` output format, so the engine transpiles the *entry* script with `format=esm`, then rewrites `import` statements to `require()` calls and wraps the rest of the body in:

```
;(async () => { /* your transpiled body */ })().then(__resolve, __reject);
```

`__resolve` and `__reject` are bindings the engine sets on the runtime to capture the final settlement. *Required-module* code (anything loaded through `require`) is transpiled with `format=cjs` and does **not** support top-level `await` — wrap in `(async () => ... )()` yourself if you need it inside a module.

## 10. Module resolution

The engine plugs a custom source loader into `goja_nodejs/require` that adds:

1. **Direct path**: if the requested file exists on disk, use it. `.ts` / `.tsx` files are transpiled on the fly.
2. **Extension fallback** when the request has no extension: try `.ts`, `.tsx`, `.js`, `.cjs`, `.mjs`, `.json` in that order.
3. **.js → .ts swap**: if `foo.js` is requested but doesn't exist, `foo.ts` (and `foo.tsx`) is tried.
4. **Directory index**: if the resolved path is a directory, try `index.ts`, `index.tsx`, `index.js`.

The base directory follows Node's CommonJS rules — relative imports resolve against the requiring module's directory, courtesy of `goja_nodejs`.

```
// All resolve to ./helpers/assert.ts:
import { check } from "./helpers/assert";
import { check } from "./helpers/assert.ts";
const { check } = require("./helpers/assert");
const { check } = require("./helpers/assert.js");
```

`node_modules` lookup *works* via the upstream registry, but the source loader doesn't currently transpile through that path — keep TS helpers under `ScriptRoot`.

## 11. Timeouts and cancellation

Two mechanisms interrupt a running script:

- `Options.Timeout` — wall-clock limit per `Run`. Exceeding it returns `scriptengine.ErrScriptTimeout`.
- `ctx` passed to `Run` / `RunFile` — cancelling the context returns `ctx.Err()` (typically `context.Canceled` or `context.DeadlineExceeded`).

Both call `vm.Interrupt(...)` and `loop.Terminate()` from a watcher goroutine. This works for **sync** JS — including `while(true){}` — because the goja VM checks the interrupt flag between bytecode steps. A host Go binding that's blocked in cgo or a long syscall isn't interruptible mid-call; if you write such a binding, plumb the engine `ctx` through to it yourself.

```
eng := scriptengine.New(scriptengine.Options{Timeout: 200 *
time.Millisecond})
_, err := eng.Run(ctx, "loop.ts", `while (true) {}`)
// err is scriptengine.ErrScriptTimeout, returned ~200–300ms after Run.
```

## 12. Error semantics

- A Go binding returning `(T, error)` automatically throws as a JS exception when `err != nil`. The exception's `.message` is `err.Error()`.

```
try { mayFail(); } catch (e) { api.log(String(e)); }
```

- A Go binding that `panic`s with a `*goja.Object` (e.g. `vm.NewGoError(...)`) does the same.
- A script throwing a JS error out of the top level surfaces from `Run` as a Go `error` whose message is the JS error's `.message`.
- Timeout errors satisfy `errors.Is(err, scriptengine.ErrScriptTimeout)`. Cancellation errors satisfy `errors.Is(err, context.Canceled)` or `context.DeadlineExceeded`.

## 13. Type generation (.d.ts)

`Engine.WriteTypes(w)` walks the registered bindings and emits a TypeScript declaration file. The mapping table (abbreviated):

| Go type                                      | TS type                                                    |
|----------------------------------------------|------------------------------------------------------------|
| <code>string</code>                          | <code>string</code>                                        |
| <code>any numeric / float64</code>           | <code>number</code>                                        |
| <code>bool</code>                            | <code>boolean</code>                                       |
| <code>[]T</code>                             | <code>T[]; []byte → Uint8Array</code>                      |
| <code>map[string]T</code>                    | <code>Record&lt;string, T&gt;</code>                       |
| <code>*T</code>                              | <code>T</code> (transparent unwrap)                        |
| <code>error</code> (single return)           | omitted (collapses to <code>void</code> )                  |
| <code>(T, error)</code> (tuple return)       | <code>T</code>                                             |
| <code>interface{}</code> / <code>any</code>  | <code>unknown</code>                                       |
| <code>goja.FunctionCall</code> parameter     | folded into <code>(...args: unknown[])</code>              |
| <code>goja.Value</code> return               | <code>unknown</code>                                       |
| <code>scriptengine.Promised[T]</code> return | <code>Promise&lt;T&gt;</code>                              |
| self-referential types                       | <code>unknown</code> (cycle detection bails after depth 4) |

```
sercon --emit-dts api.d.ts
```

In your editor, place the resulting `.d.ts` next to your scripts (or reference it via `/// <reference path="..." />`) for autocomplete.

## 14. Limitations and gotchas

---

- **No native ES modules at runtime.** All `import` is rewritten to `require` by esbuild. Side-effect-only imports run the module body exactly once per Run.
- **No top-level await inside required modules.** Wrap with `(async () => ... )()` inside the module if you need it.
- **No `tsconfig.json` honoured.** Module resolution and type checking are independent from any project-level TS config.
- **Per-Run runtime.** Side effects on `globalThis` do not persist between calls to `Run` on the same `Engine`. The `require compile` cache is shared; module `exports` are not.
- **Multi-line ESM imports may stress the entry rewriter.** The current rewriter is line-based and handles the common forms; especially gnarly inputs (comments inside the spec list, very unusual whitespace) fall back to executing as-is, which may throw.
- **RegisterConstructor is d.ts-only today.** At runtime it behaves like `Register`. True new -able constructor semantics are on the roadmap.
- **HTTP bindings are real network calls.** `api.http.*` uses `net/http` with a 5s timeout. They are not mockable from JS.

See [OUT-OF-SCOPE.md](#) for the active backlog of deferred ideas.

---

*This manual covers sercon v0.2.1. Whenever you add, remove, or change a flag, a binding, or the script API, update this file alongside the help screen (`--help`), the examples walkthrough (`--examples`), and the `CHANGELOG.md`.*